

Reshaping, Features, Transformations

Three ways to change a table — and the problem each one solves

Moussa Doumbia, Ph.D.

Department of Mathematics
Howard University

Data Wrangling and Data Engineering

Three jobs — and the problem each one solves

Job	The problem it solves
Reshape	Your tool cannot work with the layout you were given.
Engineer features	The answer is in your data, but not in any column.
Transform	The numbers are on a scale that makes your method lie.

None of the three adds information. Each one removes an **obstacle** between you and the answer.

One table. We will use it all lecture.

Quarterly sales for two stores:

store	Q1	Q2	Q3
A	100	150	250
B	400	300	300

Six numbers. Nothing is wrong with any of them.
Everything we do today is about *access*, not *correctness*.

1. Reshaping — definition

Reshaping moves the same values into a different arrangement of rows and columns.

No value is added. No value is removed.

Two shapes matter:

- **Wide** — one row per subject, one column per measurement
- **Long** — one row per *measurement*

The whole idea in one picture

WIDE (2 rows)

store	Q1	Q2	Q3
A	100	150	250
B	400	300	300



LONG (6 rows)

store	quarter	sales
A	Q1	100
A	Q2	150
A	Q3	250
B	Q1	400
B	Q2	300
B	Q3	300

$2 \times 3 = 6$ numbers on both sides. **Same facts, different shape.**

Why reshape? Reason 1 — the question gets easier

Question: average sales per quarter.

```
wide[["Q1","Q2","Q3"]].mean()      # you must NAME every column  
long.groupby("quarter").sales.mean() # you name the CONCEPT
```

Same answer today: Q1 250, Q2 225, Q3 275.

In **wide**, “quarter” lives in the *column headings* — where groupby, plot and merge cannot reach it.

In **long**, “quarter” is a *column* — an ordinary value your tools can use.

Why reshape? Reason 2 — next quarter arrives

The new file has a **Q4** column. The data really did change:

store	Q1	Q2	Q3	Q4
A	100	150	250	500
B	400	300	300	200

wide $(2, 4) \rightarrow (2, 5)$ — a new **column**

long $(6, 3) \rightarrow (8, 3)$ — two new **rows**

Now run the *exact same code* as before. Nothing in it was edited.

Same code, run again — look at the WIDE row

	before Q4 existed	after Q4 arrived
wide code	Q1 250, Q2 225, Q3 275	Q1 250, Q2 225, Q3 275 identical — Q4 never appears
long code	Q1 250, Q2 225, Q3 275	Q1 250, Q2 225, Q3 275, Q4 350 Q4 picked up automatically

The WIDE answer did not change — and that is the bug. The data gained a quarter, the answer did not, and **nothing raised an error.**

Because `["Q1","Q2","Q3"]` was typed by hand, wide code only ever sees the columns you remembered to name.

Wide → Long: melt

```
long = wide.melt(id_vars="store",  
                 var_name="quarter",  
                 value_name="sales")
```

- `id_vars` — the column(s) to **keep as is**
- `var_name` — name for the old **column headings**
- `value_name` — name for the **numbers**

Why: it turns headings into data, so `groupby`, `merge` and every plotting library can finally see them.

Long $\rightarrow$ Wide: pivot

```
wide = long.pivot(index="store",  
                  columns="quarter",  
                  values="sales")
```

Why: people read tables in two dimensions. Six rows of (store, quarter, sales) is correct but unreadable; a 2×3 grid is understood at a glance.

melt	wide $\rightarrow$ long
pivot	long $\rightarrow$ wide

Which shape do I want?

	Use it for	Because
Long	computing, grouping, plotting	machines read <i>down</i>
Wide	reporting, showing a human	people read <i>across</i>

Long to work. Wide to show.

Practice 1

Three students, three exams:

student	exam1	exam2	exam3
Ana	88	91	79
Ben	72	85	90
Cruz	95	80	84

1. After melting to long, **how many rows?** How many columns?
2. Write the `melt` call. Name the two new columns yourself.
3. Write the `pivot` call that gets you back.
4. A fourth exam is added. **Which shape needs a code change — and why?**

2. Feature engineering — definition

Feature engineering builds *new* columns from the ones you already have, so the answer becomes visible.

You add columns. **You add no new data.**

A *feature* is a column that answers a question the raw column cannot answer directly.

Why engineer features at all?

Every method you will ever use — a chart, a `groupby`,
a regression, a machine-learning model —
can only see the columns you give it.

The fact that a customer orders *on weekends* may be the whole story. But if there is no `is_weekend` column, **nothing you run will ever find it.**

The information was always in `order_date`. It just was not reachable.

Four moves that cover most of it

Move	From	To	Why
Extract	2026-03-14	month, weekday	a date hides a pattern
Combine	qty, price	revenue	the thing you care about
Compare	sales, group total	share	size $\neq$ performance
Bin	sales = 250	"medium"	groups are easier to act on

Extract, combine, compare, bin.

Example — three new columns on our long table

```
L["q_num"] = L.quarter.str[1].astype(int)
L["total"] = L.groupby("store")["sales"].transform("sum")
L["share"] = L.sales / L.total
```

store	quarter	sales	q_num	total	share
A	Q1	100	1	500	0.20
A	Q2	150	2	500	0.30
A	Q3	250	3	500	0.50
B	Q1	400	1	1000	0.40
B	Q2	300	2	1000	0.30
B	Q3	300	3	1000	0.30

Why: share removes store size from the comparison.

Why share was worth building

Store B sells more *every single quarter*. Compare the raw numbers and B wins, always.

But look at share:

- Store A is **growing** — 0.20, 0.30, 0.50
- Store B is **flat** — 0.40, 0.30, 0.30

The raw column measured **size**. The feature measures **momentum**. Those are different questions — and only one of them was in the table.

Practice 2

An online store gives you three columns:

order_date	qty	unit_price
------------	-----	------------

1. Build **three** new features — one by *extracting*, one by *combining*, one by *binning*.
2. For each, write **one sentence**: what question does it answer that the raw columns cannot?
3. Give an example of a new column that is **not** a useful feature, and say why not.

3. Transformations — definition

A **transformation** changes the *scale* of a column while keeping the *order* of the values.

Same column. Same ranking. Different numbers.

Two you will use constantly:

- **Log** — to tame a long right tail
- **Scaling** — to put different columns on one ruler

Why log? Because skew makes your detector lie

2,000 realistic transaction amounts. Nothing in them is wrong.

	skew	flagged by $ z > 3$
raw amount	5.38	37
<code>log1p(amount)</code>	0.23	3

On the raw scale you get **37 alerts** and **almost no errors**. You are detecting the *shape* of the data, not problems in it. The log removes the illusion.

How the log does it: it pulls the tail in

raw	$\log_{10}$
10	1.0
20	1.3
40	1.6
100	2.0
1,000	3.0
10,000	4.0

Raw range: a factor of **1,000**. Log range: a factor of **4**.
The order never changed — 10,000 is still the biggest.

One warning about log

```
np.log(0)      # -inf    <-- survives dropna(), destroys mean()  
np.log(-5)     # nan
```

If your column can contain **zero**, use

```
np.log1p(x)    # = log(1 + x), and log1p(0) = 0
```

Why: `-inf` is not a missing value. `dropna()` keeps it and your mean becomes `-inf`.
Check `(x <= 0).sum()` first.

Why scale? The big column decides everything

How different are these two people?

	age	income
P1	25	80,000
P2	28	30,000
difference	3	50,000
squared	9	2,500,000,000

Age is **0.0000004%** of the distance between them. Any method built on distance is really just reading income.

That is a fact about **dollars vs. years** — not about the people.

Scaling puts every column on one ruler

Our six sales numbers, mean = 250, $s = 109.54$:

sales	z-score	$\frac{x - \bar{x}}{s}$	min-max	$\frac{x - \min}{\max - \min}$
100		-1.369		0.000
150		-0.913		0.167
250		0.000		0.500
300		0.456		0.667
400		1.369		1.000

Why: after z-scoring, every column has standard deviation 1 — so no column can dominate just because its units are bigger. Order is identical.

The one rule you must not break

Compute the mean, standard deviation, min and max on the **training data only**. Then apply *those same numbers* to the test data.

Why: the test set is your stand-in for data you have never seen.

If its values helped choose your scaling numbers, then you *have* seen it — and your score is measuring the wrong thing.

Your model will look better than it is.

Practice 3

A column `donation` has values 0, 0, 25, 40, 50, 12000.

1. Would you use `np.log` or `np.log1p` here? **Why?**
2. Compute the six transformed values with the one you chose. (Two decimal places.)
3. Does the transformation change *which* value is largest? What does that tell you about what a transformation can and cannot do?
4. You split 80/20 and z-score the column. Whose mean and standard deviation do you use — and **what exactly goes wrong** if you use both halves?

The three reasons, in three sentences

1. **Reshape** — because your tools read columns, not headings.
Long to work, wide to show.
2. **Engineer features** — because a method can only see the columns you give it.
Extract, combine, compare, bin.
3. **Transform** — because scale changes the answer, and units are not meaning.
Log for the tail, scaling for the ruler.

None of the three invents information. Each one makes information you **already had** reachable.

Cheat sheet

```
# reshape -- so tools can see the variable
long = wide.melt(id_vars=, var_name=, value_name=)
wide = long.pivot(index=, columns=, values=)

# features -- so the question has a column to live in
df["month"]    = df.date.dt.month                # extract
df["revenue"]  = df.qty * df.price                # combine
df["total"]    = df.groupby("k")["v"].transform("sum") # compare
df["band"]     = pd.cut(df.v, bins=[0,150,300,np.inf]) # bin

# transform -- so scale stops distorting the answer
df["log_v"]    = np.log1p(df.v)                  # tail
df["z"]        = (df.v - df.v.mean()) / df.v.std(ddof=1) # ruler
```

Reshape. Add. Rescale.

You are not changing the facts.

You are removing what stands between you and them.

Moussa Doumbia, Ph.D. · Department of Mathematics, Howard University